

BLOK · CORE REDESIGN

Rewriting the core of Blok

Blok core stays one TypeScript orchestration language but trades every stringly-typed surface — \$.state refs, js/ templates, raw ctx branch conditions, the Nodes.ts registry, and ~10 lockstep packages — for typed step handles over the unchanged auto-persistence engine, shipped as one @blokjs/core package.

Old → new, with complete code · workflows · nodes · package · context · persistence · 7 runtimes

The new model at a glance

Everything that changes, in one place. Details and full code follow.

- Authoring becomes typed handles: `step("id", node, inputs)` returns a value shaped like the node's Zod output; reading a field records a structural `{ $ref:{step,path} }`. No `$`, no `js/`, no `ctx` anywhere an author types — those become internal runtime plumbing.
- The engine does not move. Auto-persist-every-output-to-state[id]-on-success (`PersistenceHelper.applyStepOutput`, Rules 0/2/3 verified) and the Mapper that resolves refs against `state[step]` are byte-identical; the handle is just the typed surface over them. Identical runtime, checkable surface.
- `defineNode()` is untouched and `Nodes.ts` is deleted: local TS nodes are imported straight into the workflow (the import IS the registration, carrying the Zod output type into the handle); cross-runtime nodes get a generated `runtimeNode` stub from the runtime's `ListNodes` manifest (`input_schema_json/output_schema_json`, verified in `runtime.proto`). One mental model for all 7 runtimes.
- Branch conditions and string interpolation stop being untyped: `branch` takes a boolean handle or a typed op, `tpl` keeps refs structural inside strings. Today's `eq/gt/lt` are confirmed string builders (return type **string**) — the new ops read real handle fields, so a renamed output fails to compile instead of silently evaluating to undefined (the verified `countries-vs-facts.json` raw-when footgun).
- Packaging collapses ~10 lockstep `@blokjs/*` deps into one `@blokjs/core` (runner + shared + helper DSL), killing the `@blokjs/helper-vs-helpers` one-letter trap. Nodes stay independent semver packages; triggers like `http` are opt-in submodules.
- The IR is the lingua franca: the `.ts` DSL compiles to language-neutral `{ use, type:"runtime.x", inputs:{ $ref } }` JSON that the canvas, JSON twin, and AI all consume — same IR and same `GrpcRuntimeAdapter` dispatch as today, whether the work runs in-process in TS or out-of-process in Python.
- Net author-visible surface shrinks from `{ctx.request/req/prev/response/state/vars/error, $, js/ strings, .data envelopes, raw when}` to exactly `{step handles, req, tpl, eq/gt/lt, forEach}`.

01 Workflows: from \$.state strings to typed handles

Today a Blok workflow is a flat array of step objects, and every cross-step data reference is a hand-typed string the compiler never sees:

``$.state.validate.productId`` for inputs, ``js\...${ctx.state...}\`` for interpolation, and a raw ``ctx.*`` JS string for branch conditions. A typo, a renamed output field, or a wrong alias all fail silently at runtime. The new handle syntax replaces every string with a typed value: ``step(id, node, inputs)`` returns a handle shaped like the node's Zod output, references are real property reads, ``tpl`` keeps interpolation structural, and ``branch`` takes a boolean handle (or a ``gt`/`eq`/`op``) instead of a raw string. Same auto-persistence, same if-else engine underneath — just a surface the type system can actually check.

● A multi-step workflow that fetches data and chains it forward

● BEFORE — today

`$.state.<id>` strings and `js/` interpolation — the step id and every reference to it can silently drift apart

```
import { workflow, $ } from "@blokjs/helper";

export default workflow({
  name: "order-intake",
  version: "1.0.0",
  trigger: {
    http: { method: "POST", path: "/orders", accept: "application/json" },
  },
  steps: [
    {
      id: "validate",
      use: "validate-order",
      type: "module",
      inputs: { body: $.req.body },
    },
    {
      id: "checkStock",
      use: "@blokjs/api-call",
      type: "module",
      inputs: {
        // js/ string interpolation — productId is a magic substring. Typo it
```

```

    // (productID) and you get `undefined` at runtime, never a compile error.
    url: "js/`https://inv.api/stock/${ctx.state.validate.productId}`",
    method: "GET",
  },
},
{
  id: "createOrder",
  use: "create-order",
  type: "module",
  inputs: {
    // Three cross-step refs, each a hand-typed string keyed off a step id
    // above. Nothing checks that `validate` / `checkStock` exist or that
    // `.productId` / `.qty` are real output fields.
    productId: "$.state.validate.productId",
    qty: "$.state.validate.qty",
    stock: "$.state.checkStock",
  },
},
{
  id: "respond",
  use: "@blokjs/respond",
  type: "module",
  inputs: { status: 201, body: "$.state.createOrder" },
  ephemeral: true,
},
],
});

```

● AFTER — proposed

Typed handles: `step(id, node, inputs)` returns the node's output shape; references are real property reads — autocompleted and refactor-safe

```

import { workflow, step, http, tpl } from "@blokjs/core";
import validateOrder from "./nodes/validate-order";
import createOrder from "./nodes/create-order";
import httpGet from "@blokjs/api-call";
import respond from "@blokjs/respond";

```

```

export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  // `validate` is a typed handle shaped like validateOrder's Zod output.
  const validate = step("validate", validateOrder, { body: req.body });

  // tpl keeps the ref structural inside the string – validate.productId is a
  // real typed read, not a magic substring. Rename the output field and this
  // line fails to compile instead of resolving to undefined at runtime.
  const stock = step("checkStock", httpGet, {
    url: tpl`https://inv.api/stock/${validate.productId}`,
    method: "GET",
  });

  // Cross-step reads are just property accesses on the handles. Auto-persisted
  // to state under the hood – usable anywhere downstream, no save step.
  const order = step("createOrder", createOrder, {
    productId: validate.productId,
    qty: validate.qty,
    stock,
  });

  return step("respond", respond, { status: 201, body: order }, { ephemeral: true });
});

```

• The same workflow with a branch on a fetched value (the raw-ctx footgun)

● BEFORE — today

branch.when is a raw JS string the if-else node evals against ctx — wrong alias or a \$ proxy here silently mis-evaluates; the eq() escape hatch is still an untyped string builder

```

import { workflow, branch, $ } from "@blokjs/helper";

export default workflow({
  name: "order-intake",
  version: "1.0.0",
  trigger: {
    http: { method: "POST", path: "/orders", accept: "application/json" },

```

```
},
steps: [
  {
    id: "validate",
    use: "validate-order",
    type: "module",
    inputs: { body: $.req.body },
  },
  {
    id: "checkStock",
    use: "@blokjs/api-call",
    type: "module",
    inputs: {
      url: "js/`https://inv.api/stock/${ctx.state.validate.productId}`",
      method: "GET",
    },
  },
],
branch({
  id: "route",
  // FOOTGUN: `when` is NOT mapper-resolved – the if-else node compiles it
  // into a raw Function(ctx, ...). The canonical idiom is a hand-written
  // raw-ctx string, spelled against the right alias (ctx.state.<id>) and
  // unchecked by the compiler. Pass a $ proxy here (which compiles to
  // "js/ctx....") and the branch evaluates that literal string as truthy.
  // The eq()/gt() helpers exist to escape this – but they still take a
  // $ proxy you have to spell correctly, with no type behind it.
  when: "ctx.state.checkStock.inStock === true",
  then: [
    {
      id: "createOrder",
      use: "create-order",
      type: "module",
      inputs: {
        productId: "$.state.validate.productId",
        qty: "$.state.validate.qty",
        stock: "$.state.checkStock",
      },
    },
  ],
},
```

```

    },
    {
      id: "ok",
      use: "@blokjs/respond",
      type: "module",
      inputs: { status: 201, body: "$.state.createOrder" },
      ephemeral: true,
    },
  ],
  else: [
    {
      id: "no",
      use: "@blokjs/respond",
      type: "module",
      inputs: { status: 409, body: { error: "out of stock" } },
      ephemeral: true,
    },
  ],
},
}),
],
});

```

● AFTER — proposed

branch takes a boolean handle (or a typed gt/eq op over handle fields) — the condition is the node's actual output, type-checked, with no raw ctx string anywhere

```

import { workflow, step, branch, http, tpl, gt } from "@blokjs/core";
import validateOrder from "./nodes/validate-order";
import createOrder from "./nodes/create-order";
import httpGet from "@blokjs/api-call";
import respond from "@blokjs/respond";

export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  const validate = step("validate", validateOrder, { body: req.body });

  const stock = step("checkStock", httpGet, {
    url: tpl`https://inv.api/stock/${validate.productId}`,
  });
});

```

```

    method: "GET",
  });

  // stock.inStock is a typed boolean handle. branch reads it as a structural
  // ref – no raw-ctx string, no $ proxy, no footgun. The if-else node still
  // does the work under the hood; the author never sees ctx.
  return branch(stock.inStock, {
    then: () => {
      const order = step("createOrder", createOrder, {
        productId: validate.productId,
        qty: validate.qty,
        stock,
      });
      return step("ok", respond, { status: 201, body: order }, { ephemeral: true });
    },
    else: () =>
      step("no", respond, { status: 409, body: { error: "out of stock" } }, { ephemeral: true }),
  });
});

// For non-boolean conditions, typed comparison ops replace the raw string –
// the operands are real handle fields, so a renamed output is a compile error:
// branch(gt(stock.available, validate.qty), { then: () => ..., else: () => ... });

```

TAKEAWAYS

- One orchestration language stays: workflows are TypeScript. The 7 runtimes are for nodes (the work units), not orchestration — moving to all-.ts authoring loses nothing.
- `step("id", node, inputs)` returns a typed handle shaped like the node's Zod output. Reading `validate.productId` records a structural `{ $ref: { step, path } }` — a real property access, autocompleted and refactor-safe, not a magic string.
- Every step's output is already auto-persisted to `state[id]` on success. The handle is just the typed surface over that — referenceable anywhere downstream, no manual save step. `ephemeral:true` still opts side-effect steps out.
- `branch` takes a boolean handle (`stock.inStock`) or a typed op (`gt(stock.available, validate.qty)`). Today's escape hatch — the `eq()/gt()` comparators — still hand you an untyped string builder over a `$` proxy; the new ops read real handle fields, so a renamed output fails to compile. `tpl` does the same for string interpolation.

- No \$, no js/, no ctx anywhere in authoring. ctx.state / ctx.prev / ctx.request become internal runtime plumbing; the mapper resolves the same { \$ref } against state[step] using today's engine — cleaner surface, identical runtime.

02 Nodes + the death of Nodes.ts

The node itself was never the problem — `defineNode()` (Zod in, Zod out, a pure `execute`) stays byte-for-byte the same. What changes is everything *around* it: today a hand-maintained `Nodes.ts` registry maps magic strings like `"@blokjs/api-call"` to imported nodes — typed `Record<string, BlokService<unknown>>`, where the `unknown` erases every output type — and workflows reference nodes by re-typing that same string (use: `"@blokjs/api-call"`) with zero type-checking. In the new model that registry file is deleted: a local TS node is imported straight into the workflow and passed to `step()` by value, so the node's Zod output type flows directly into the handle. Cross-runtime nodes (Go/Python/...) keep the language-neutral `{ use, type: "runtime.x" }` IR, but it's now emitted from a generated typed stub instead of a string an author guessed.

● The node definition — unchanged across old and new

● BEFORE — today

`defineNode()` — Zod in, Zod out, pure execute. Nothing here was ever broken.

```
// nodes/web/api-call@1.0.0/index.ts
//
// defineNode() — function-first, Zod in / Zod out, pure execute.
// This is the REAL api-call node shipped with Blok.

import { defineNode } from "@blokjs/runner";
import type { JsonLikeObject } from "@blokjs/runner";
import { z } from "zod";
import { runApiCall } from "../util";

export default defineNode({
  name: "api-call",
  description: "Makes HTTP API calls with automatic JSON handling",

  input: z.object({
    url: z.string().url("Must be a valid URL"),
    method: z.string().default("GET"),
    headers: z.record(z.string()).optional().default({}),
    body: z.record(z.unknown()).optional().default({}),
    responseType: z.string().optional().default("json"),
  }),
```

```

output: z.union([
  z.string(),           // text response
  z.record(z.unknown()), // JSON response
]),

async execute(ctx, input) {
  // Fallback to the previous step's data when no body is passed.
  const body =
    Object.keys(input.body).length > 0
    ? (input.body as JsonLikeObject)
    : (ctx.response.data as JsonLikeObject);

  const result = await runApiCall(
    input.url,
    input.method,
    input.headers as JsonLikeObject,
    body,
    input.responseType,
  );

  return result;
},
});

```

● AFTER — proposed

defineNode() is untouched — same Zod schemas, same execute. Only the import package consolidates (@blokjs/runner → @blokjs/core).

```

// nodes/web/api-call@1.0.0/index.ts
//
// IDENTICAL logic. The import package (@blokjs/core) is the only line
// that moves, and that belongs to the package-consolidation dimension,
// not this one. The ctx.response.data fallback stays — it's runtime
// plumbing INSIDE a node, not authoring surface.

import { defineNode } from "@blokjs/core"; // was: "@blokjs/runner"

```

```
import type { JsonLikeObject } from "@blokjs/core";
import { z } from "zod";
import { runApiCall } from "../util";

export default defineNode({
  name: "api-call",
  description: "Makes HTTP API calls with automatic JSON handling",

  input: z.object({
    url: z.string().url("Must be a valid URL"),
    method: z.string().default("GET"),
    headers: z.record(z.string()).optional().default({}),
    body: z.record(z.unknown()).optional().default({}),
    responseType: z.string().optional().default("json"),
  }),

  output: z.union([
    z.string(),
    z.record(z.unknown()),
  ]),

  async execute(ctx, input) {
    const body =
      Object.keys(input.body).length > 0
        ? (input.body as JsonLikeObject)
        : (ctx.response.data as JsonLikeObject);

    const result = await runApiCall(
      input.url,
      input.method,
      input.headers as JsonLikeObject,
      body,
      input.responseType,
    );

    return result;
  }
});
```

```
},  
});
```

• The registry — a hand-maintained file vs. a direct import

● BEFORE — today

Every node double-registered: once in the map, again as a string in each workflow — all typed `unknown`.

```
// triggers/http/src/Nodes.ts  
//  
// THE REGISTRY. Every node a workflow can `use:` has to be imported  
// here and mapped to its magic string. New node => edit this file.  
// The map is `Record<string, BlokService<unknown>>` — the `unknown`  
// is where every output type is erased.  
  
import ApiCall from "@blokjs/api-call";  
import { HELPER_NODES } from "@blokjs/helpers";  
import IfElse from "@blokjs/if-else";  
import type { BlokService } from "@blokjs/runner";  
import ChainInit from "./nodes/chain-init/index";  
import ChainVerify from "./nodes/chain-verify/index";  
import EvalNodes from "./nodes/eval/index";  
import ExampleNodes from "./nodes/examples/index";  
import RuntimeBridge from "./nodes/runtime-bridge/index";  
  
const nodes: Record<string, BlokService<unknown>> = {  
  "@blokjs/api-call": ApiCall,  
  "@blokjs/if-else": IfElse,  
  // v0.5 generic helpers: expr, ctx-publish, throw, log, audit-log,  
  // in-memory-kv, json-schema, etc. Registered globally so any  
  // workflow can use them via `use: "@blokjs/<name>`".  
  ...(HELPER_NODES as unknown as Record<string, BlokService<unknown>>),  
  "chain-init": ChainInit,  
  "chain-verify": ChainVerify,  
  "runtime-bridge": RuntimeBridge,  
  ...(EvalNodes as unknown as Record<string, BlokService<unknown>>),  
  ...ExampleNodes,
```

```
};

export default nodes;

// — and in a workflow, the node is referenced by re-typing the string —
// {
//   id: "fetch",
//   use: "@blokjs/api-call", // <- magic string, no type-check
//   inputs: { url: "$.req.body.url" },
// }
// Typo "@blokjs/api-cal" => "Node type X not found" at RUN time.
```

● AFTER — proposed

Import the node, pass it to step() by value. The registry file is gone; the node's output type flows into the handle and any handle is referenceable downstream.

```
// triggers/http/src/Nodes.ts — DELETED.
//
// There is no registry file. A local TS node is the value you already
// import; you hand it to step() directly. No string, no map entry.

// workflows/orders/intake.ts
import { workflow, step, branch, http, tpl } from "@blokjs/core";
import apiCall      from "@blokjs/api-call"; // the node, by value
import validateOrder from "../../nodes/validate-order";
import createOrder  from "../../nodes/create-order";
import respond      from "@blokjs/respond";

export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  const validate = step("validate", validateOrder, { body: req.body });

  // step(id, node, inputs) -> typed handle shaped like the node's Zod output.
  // `stock.inStock` is checked against api-call's output type at COMPILE time.
  const stock = step("checkStock", apiCall, {
    url: tpl`https://inv.api/stock/${validate.productId}`,
  });
});
```

```

return branch(stock.inStock, {
  then: () => {
    // `stock` was declared OUTSIDE this arm — auto-persistence means its
    // handle is referenceable anywhere downstream, no manual save step.
    const order = step("createOrder", createOrder, {
      productId: validate.productId,
      qty: validate.qty,
      stock,
    });
    return step("ok", respond, { status: 201, body: order });
  },
  else: () =>
    step("no", respond, { status: 409, body: { error: "out of stock" } })),
});
});

// Rename the import, the node still resolves — it's a value, not a string.
// Delete the node file and `validateOrder` is a RED SQUIGGLE before you save,
// not a "Node type X not found" at run time.

```

● Cross-runtime nodes — the one string that survives, now generated

● BEFORE — today

Out-of-process nodes are pure strings — name, runtime kind, and output shape all unchecked until the run.

```

// OLD: a Go / Python node is referenced the same untyped way.
// The registry can't even hold it (it lives in another process), so the
// author writes the runtime kind AND the name as hand-typed strings.

// triggers/http/workflows/json/enrich.json
{
  "name": "Enrich Lead",
  "version": "1.0.0",
  "trigger": { "http": { "method": "POST", "path": "/enrich" } },
  "steps": [
    {
      "id": "score",

```

```

    "use": "lead-scorer",          // name must match the Python manifest
    "type": "runtime.python3",    // hand-typed runtime kind
    "inputs": { "email": "$.req.body.email" }
  },
  {
    "id": "respond",
    "use": "@blokjs/respond",
    "inputs": { "body": "$.state.score" } // score's shape: unknown
  }
]
}

```

*// Nothing checks that "lead-scorer" exists, that it runs on python3,
 // or that `score` has a `riskBand` field. All three are run-time roulette.*

● AFTER — proposed

A generated typed stub gives Go/Python nodes the same compile-time handle as local TS — the runtime string is emitted, never hand-written.

```

// NEW: codegen reads the node's manifest/JSON-Schema (the same pipeline
// that already emits register_user_nodes.go) and writes a typed stub.
// You import the stub like any node; the { use, type } IR is DERIVED from it.

// .blok/generated/nodes/lead-scorer.ts (generated — do not edit)
import { runtimeNode } from "@blokjs/core";

export default runtimeNode<
  { email: string }, // input (from manifest)
  { score: number; riskBand: "low" | "med" | "high" } // output (from manifest)
>("lead-scorer", "runtime.python3");

// workflows/enrich.ts
import { workflow, step, http } from "@blokjs/core";
import leadScorer from "../blok/generated/nodes/lead-scorer";
import respond from "@blokjs/respond";

export default workflow("enrich-lead", { trigger: http.post("/enrich") }, (req) => {

```



```

const score = step("score", leadScorer, { email: req.body.email });
// score.riskBand is "low" | "med" | "high" at COMPILE time -
// a Python node, fully typed on the TS authoring side.
return step("respond", respond, { body: score });
});

// Compiles to the SAME language-neutral IR the canvas + JSON twin read.
// `req` is the typed trigger handle; reading `req.body.email` records a ref:
//   { id: "score", use: "lead-scorer", type: "runtime.python3",
//     inputs: { email: { $ref: { step: "@trigger", path: ["body","email"] } } } }
// The string still exists in the IR - but no human ever typed it.

```

TAKEAWAYS

- `defineNode()` does not change — Zod input/output and a pure `execute` were never the friction. The only edit to a node file is the package the import comes from (`@blokjs/runner` → `@blokjs/core`), which belongs to the packaging dimension. The node-internal `ctx.response.data` fallback stays — that's runtime plumbing inside a node, not the author-facing surface the new model replaces.
- `triggers/http/src/Nodes.ts` is deleted. Today it's a hand-maintained `Record<string, BlokService<unknown>>` that erases every node's output type and forces a second edit (plus a magic string in every workflow) per node. A local TS node is now imported straight into the workflow and passed to `step()` by value.
- The `use:` magic string disappears for local nodes. `step("id", apiCall, {...})` carries the node's Zod output type into the handle, so `stock.inStock` is checked at compile time and a deleted/renamed node is a red squiggle before save — not a "Node type X not found" at run time.
- Auto-persistence is what makes the handle work anywhere: every step's output lands in `state[id]` on success, so a handle declared outside a branch (like `stock`) is referenceable inside it with no manual save step. The handle is just the typed surface over Blok's existing default-store.
- Cross-runtime nodes (Go/Rust/Python/...) keep the language-neutral `{ use, type:"runtime.x" }` IR — the lingua franca the canvas and JSON twin consume — but it's now emitted from a generated typed stub (`runtimeNode<In,Out>(…)`) produced from the node's manifest, the same codegen that already writes `register\_user\_nodes.go`. A Python node becomes fully typed on the TS authoring side. One mental model for every node: import it, hand it to `step()`, read its handle — whether the work runs in-process in TS or out-of-process in Python is an IR detail, not an authoring concern.

03 The unified @blokjs/core package

Today a Blok project wires together ~10 separately-versioned `@blokjs/\*` packages, all locked to the same monorepo version, plus a hand-maintained `Nodes.ts` registry that `blokctl install` regex-patches in place. The two most-imported packages are named `@blokjs/helper` (the DSL) and `@blokjs/helpers` (a bag of nodes) — a one-letter trap that bites everyone. The redesign collapses the framework into ONE package: `import { workflow, step, branch, defineNode, http } from "@blokjs/core"`. Nodes become ordinary npm packages you import directly (no `Nodes.ts` registry), and triggers like `http` are opt-in submodules — so the dependency list shrinks from a lockstep cluster to a single framework dep plus exactly the nodes you actually use.

• The imports + node registry a scaffolded project carries today vs. one framework import

● BEFORE — today

~10 @blokjs/* packages, a hand-patched Nodes.ts map, the helper/helpers trap, and three stringly-typed ref dialects (\$. / js/ / raw-ctx when)

```
// — triggers/http/src/Nodes.ts —————
// Hand-maintained registry: a Record<string, BlokService>. Every
// used node must be imported AND added to the map, or it silently
// fails to resolve at runtime. `blokctl install node` regex-patches
// this exact file (breaks if you reformat it).
import ApiCall from "@blokjs/api-call";
import { HELPER_NODES } from "@blokjs/helpers"; // the NODES bag – plural
import IfElse from "@blokjs/if-else";
import type { BlokService } from "@blokjs/runner";
import ChainInit from "../nodes/chain-init/index";
import ChainVerify from "../nodes/chain-verify/index";

const nodes: Record<string, BlokService<unknown>> = {
  "@blokjs/api-call": ApiCall,
  "@blokjs/if-else": IfElse,
  ...(HELPER_NODES as unknown as Record<string, BlokService<unknown>>),
  "chain-init": ChainInit,
  "chain-verify": ChainVerify,
};
```

```
export default nodes;

// — triggers/http/src/workflows/order-intake.ts —————
// The DSL lives in @blokjs/helper – SINGULAR. One letter off from
// the @blokjs/helpers nodes bag above; mistype it and the error is
// cryptic. Refs are stringly-typed: $.state.<id> for cross-step
// reads, js/`...` to interpolate a ref inside a string, and the
// branch `when` must be a RAW ctx.* expression (a $.-prefixed when
// 500s at runtime – typecheck won't catch it).
import { workflow, branch, $ } from "@blokjs/helper";

export default workflow({
  name: "order-intake",
  version: "1.0.0",
  trigger: { http: { method: "POST", path: "/orders" } },
  steps: [
    { id: "validate", use: "validate-order", inputs: { body: $.req.body } },
    { id: "checkStock", use: "@blokjs/api-call",
      inputs: { url: "js/`https://inv.api/stock/${ctx.state.validate.productId}`" } },
    branch({
      id: "route",
      when: "ctx.state.checkStock.inStock === true",
      then: [
        { id: "createOrder", use: "create-order",
          inputs: { productId: $.state.validate.productId, qty: $.state.validate.qty } },
        { id: "ok", use: "@blokjs/respond",
          inputs: { status: 201, body: $.state.createOrder }, ephemeral: true },
      ],
      else: [
        { id: "no", use: "@blokjs/respond",
          inputs: { status: 409, body: { error: "out of stock" } }, ephemeral: true },
      ],
    }),
  ],
});
```

● AFTER — proposed

One import from @blokjs/core; nodes imported directly; one typed-handle surface replaces \$., js/, raw-ctx when, and Nodes.ts

```
// — workflows/order-intake.ts —————
// ONE framework import. No Nodes.ts registry – local nodes are
// imported directly; the import IS the registration. Refs are
// typed handles, so there is no $. / js/ / raw-ctx split to learn.
import { workflow, step, branch, http, tpl, gt } from "@blokjs/core";

// Local TS nodes: plain packages, imported where used.
import { validateOrder, createOrder } from "@acme/order-nodes";
import { httpGet, respond } from "@blokjs/stdlib-nodes";

export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  const validate = step("validate", validateOrder, { body: req.body });
  const stock = step("checkStock", httpGet, {
    url: tpl`https://inv.api/stock/${validate.productId}`,
  });
  return branch(gt(stock.inStock, false), {
    then: () => {
      const order = step("createOrder", createOrder, {
        productId: validate.productId,
        qty: validate.qty,
        stock,
      });
      return step("ok", respond, { status: 201, body: order });
    },
    else: () => step("no", respond, { status: 409, body: { error: "out of stock" } }),
  });
});
```

● The project's package.json dependency list — lockstep cluster vs. one framework dep + the nodes you use

● BEFORE — today

10 lockstep-versioned deps; helper vs helpers side by side; framework and nodes can't move independently

```
{
  "name": "my-blok-app",
  "version": "0.0.1",
  "dependencies": {
    "@blokjs/runner": "^0.7.0",
    "@blokjs/shared": "^0.7.0",
    "@blokjs/helper": "^0.7.0",
    "@blokjs/helpers": "^0.7.0",
    "@blokjs/trigger-http": "^0.7.0",
    "@blokjs/trigger-worker": "^0.7.0",
    "@blokjs/api-call": "^0.7.0",
    "@blokjs/if-else": "^0.7.0",
    "@blokjs/respond": "^0.7.0",
    "zod": "^3.23.0"
  }
}
```

// Every @blokjs/ dep is pinned to the SAME monorepo version (0.7.0).
 // Each node re-declares @blokjs/runner + @blokjs/shared as its OWN
 // deps, also locked to that version, so a runner patch forces a
 // coordinated bump of ALL of them – runner, shared, both helper
 // packages, both triggers, and every node. You cannot upgrade the
 // framework without re-releasing the nodes, and a version drift
 // surfaces as duplicate-install bugs.*

● AFTER — proposed

One framework dep + only the node packages you use; triggers opt-in; nodes on independent semver

```
{
  "name": "my-blok-app",
  "version": "0.0.1",
  "dependencies": {
    "@blokjs/core": "^1.0.0",
    "@blokjs/stdlib-nodes": "^1.0.0",
    "@acme/order-nodes": "^0.3.0",
    "zod": "^3.23.0"
  }
}
```

```

}

// @blokjs/core = runner + shared + types + DSL, ONE versioned package,
// ONE import. Triggers (http, worker) ship as opt-in submodules of core
// - pulled in via `import { http } from "@blokjs/core"`, not separate deps.
// NODES are ordinary packages on their own semver: upgrade the framework
// without re-releasing a single node, and pin/upgrade each node bag on its
// own line. Old @blokjs/runner / @blokjs/helper / @blokjs/helpers names
// remain as deprecated re-export shims, so existing projects keep building.

```

TAKEAWAYS

- One import replaces ~10: `@blokjs/core` folds today's `@blokjs/runner` + `@blokjs/shared` + `@blokjs/helper` (the DSL) into a single versioned framework package, so `import { workflow, step, branch, defineNode, http } from "@blokjs/core"` is the whole framework surface — nodes stay separate packages.
- The `@blokjs/helper` (DSL) vs `@blokjs/helpers` (nodes bag) one-letter trap is gone — the DSL is part of core, and nodes are explicit, directly-imported packages, not a `HELPER\_NODES` grab-bag spread into a registry map.
- `Nodes.ts` is deleted: local TS nodes are imported straight into the workflow (the import IS the registration), and cross-runtime nodes get a generated typed stub — no hand-patched `Record<string, BlokService>` that `blokctl install` regex-edits and silently drops a node from.
- Three stringly-typed ref dialects collapse to one: today authors juggle `\${state.<id>}` for reads, `js\`...\`` to interpolate a ref into a string, and a RAW `ctx.\*` expression for `branch` `when` (a `\${.-`-prefixed `when` compiles fine but 500s at runtime). Handles plus `tpl`/`gt` give one typed surface with no `\${`, `js/`, or `ctx` anywhere.
- Lockstep release ends: today every `@blokjs/\*` dep pins the same monorepo version and each node re-declares runner/shared as its own deps, so a runner patch forces a coordinated re-release of everything. With core as one dep and nodes on independent semver — behind deprecated re-export shims for the old names — the framework and each node package move on their own.

04 Context & passing data between steps

Today a Blok author has to learn the runtime's plumbing to move data between steps: five overlapping ctx surfaces (ctx.state / ctx.prev / ctx.vars / ctx.response / ctx.request), a \$ proxy that secretly compiles to js/ctx.* strings, node outputs that arrive wrapped in a .data envelope, and a branch condition that is a raw ctx string the typechecker never sees. The new model exposes exactly two things to authors — typed step handles and the typed req — and the entire ctx.* surface becomes invisible runtime plumbing. Auto-persistence is unchanged underneath, so reading a far-upstream step is the same one-liner as reading the adjacent one.

- A real multi-step workflow that threads request data, an adjacent output, and a far-upstream output — then branches on it

● BEFORE — today

Five ctx surfaces, \$ that hides js/ strings, a .data envelope to unwrap, and a raw-string branch the compiler can't check

```
import { workflow, branch, $ } from "@blokjs/helper";

export default workflow({
  name: "Order Intake",
  version: "1.0.0",
  trigger: { http: { method: "POST", path: "/orders" } },
  steps: [
    {
      id: "validate",
      use: "validate-order",
      // $.req.body compiles to the string "js/ctx.req.body" at definition time.
      inputs: { body: $.req.body },
    },
    {
      id: "checkStock",
      use: "@blokjs/api-call",
      // $.state.validate reads the ADJACENT step via ctx.state. You must know
      // the id matches the string the runner stored the output under, AND that
      // the parsed body lives under `.data` once api-call returns.
      inputs: {
        url: "js`https://inv.api/stock/${ctx.state.validate.productId}`",
      },
    },
  ],
});
```

```

    method: "GET",
  },
},
branch({
  id: "route",
  // FOOTGUN: `when` is a RAW ctx string. NOT the $ proxy, NOT mapper-resolved.
  // Write $.state.checkStock.inStock here and it silently evaluates to
  // undefined -> always takes the else arm. The typechecker sees nothing.
  // (The shipped countries-vs-facts.json uses exactly this raw form:
  //  when: 'ctx.request.query.countries === "true"'.)
  when: "ctx.state.checkStock.data.inStock === true",
  then: [
    {
      id: "createOrder",
      use: "create-order",
      inputs: {
        // FAR-UPSTREAM read: createOrder needs `validate`, two steps back.
        // ctx.prev here is `checkStock` (overwritten every step) so it is
        // USELESS for this – you must reach back into ctx.state by id.
        productId: "js/ctx.state.validate.productId",
        qty: "js/ctx.state.validate.qty",
        // ...and the adjacent stock result, reaching through `.data` again.
        stock: "js/ctx.state.checkStock.data",
      },
    },
  ],
  {
    id: "ok",
    use: "@blokjs/respond",
    ephemeral: true,
    // $.state.createOrder vs js/ctx.state.createOrder vs ctx.vars.createOrder
    // are all the same object under three names. Which do you reach for?
    inputs: { status: 201, body: $.state.createOrder },
  },
],
else: [
  {
    id: "no",
  },
]

```



```

        use: "@blokjs/respond",
        ephemeral: true,
        inputs: { status: 409, body: { error: "out of stock" } },
      },
    ],
  },
],
});

```

● AFTER — proposed

Two surfaces only — typed handles + req; no envelope, no id strings, far-upstream reads are one-liners

```

import { workflow, step, branch, http, tpl, eq } from "@blokjs/core";
import { validateOrder, createOrder } from "./nodes"; // local TS nodes, imported directly
import { httpGet, respond } from "@blokjs/stdlib";

```

```

export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  // req is the typed trigger handle — req.body is shaped by the trigger, no ctx.request.
  const validate = step("validate", validateOrder, { body: req.body });

```

```

  // Reading validate.productId records a structural ref to step "validate".
  // tpl keeps the ref structural inside the string — no js/`...` template.

```

```

  const stock = step("checkStock", httpGet, {
    url: tpl`https://inv.api/stock/${validate.productId}`,
    method: "GET",
  });

```

```

  // eq() is a typed comparison op -> branch.when is never a raw string.
  // stock.inStock is type-checked against httpGet's Zod output — no .data
  // envelope to remember, the handle is shaped like the node's output.

```

```

  return branch(eq(stock.inStock, true), {
    then: () => {

```

```

      // FAR-UPSTREAM read is identical to an adjacent read: just use the handle.
      // validate is two steps back; createOrder reads it with no ctx.state, no id
      // string to keep in sync, no "is this prev or state?" question.

```

```

      const order = step("createOrder", createOrder, {
        productId: validate.productId,

```

```

    qty: validate.qty,
    stock,                // pass the whole upstream handle as one input
  });
  return step("ok", respond, { status: 201, body: order });
},
else: () => step("no", respond, { status: 409, body: { error: "out of stock" } })),
});
});

```

● Every old ctx.* / \$ / js/ form and the single handle form that replaces it

● BEFORE — today

Three names for the same object, two ways to write each, one of them an unchecked string

// ---- THE OLD AUTHORIZING SURFACE (everything below is something an author had to learn) ----

```

ctx.request.body           // trigger payload (also aliased ctx.req.body)
ctx.req.params.id          // same object, second name
$.req.query.page           // $ proxy -> compiles to "js/ctx.req.query.page" verbatim
                           // (the proxy does NOT rewrite req->request; it emits what you type)

ctx.prev.data              // the IMMEDIATELY previous step's output envelope
ctx.response.data          // same object as ctx.prev, third name
$.prev.data                // -> "js/ctx.prev.data"

ctx.state.validate.productId // a NAMED step's output, by id string
ctx.vars.validate.productId  // same object as ctx.state, alias
$.state.validate.productId   // -> "js/ctx.state.validate.productId"
"js/ctx.state.checkStock.data" // the hand-written wire string the $ proxy emits

branch({ when: "ctx.state.checkStock.data.inStock === true" }) // RAW ctx string, no $
$error.message           // captured error inside tryCatch.catch -> "js/ctx.error.message"

```

● AFTER — proposed

One typed handle collapses state/vars/prev/response into the same thing; req covers request

```
// ---- THE NEW AUTHORIZING SURFACE (handles + req; ctx.* is now invisible runtime plumbing) ----

req.body                // trigger payload – typed, replaces ctx.request / ctx.req / $.req
req.params.id
req.query.page

const v = step("validate", validateOrder, { body: req.body });
v.productId             // adjacent OR far-upstream – same handle, replaces ctx.prev / ctx.response / ctx.state / ctx.vars
v                       // pass the whole output as one input (was ctx.state.validate)

tpl`.../${v.productId}` // structural ref inside a string – replaces js/`...` templates

branch(eq(stock.inStock, true), { ... }) // typed op – replaces the raw `when` string; compiler checks it

tryCatch({ try: () => ..., catch: (err) => step("log", logError, { msg: err.message }) })
// err is a typed handle – replaces $.error / ctx.error

// MAPPING TABLE
// ctx.request.* / ctx.req.* / $.req.*      -> req.*
// ctx.prev.* / ctx.response.* / $.prev.*    -> <handle>          (adjacent step's const)
// ctx.state.<id>.* / ctx.vars.<id>.* / $.state.<id>.* -> <handle>.*      (any step, near or far)
// "js/ctx.state.<id>.x" wire string           -> <handle>.x        (no string, typed)
// js/`...${ctx.state.x}...`                  -> tpl`...${handle.x}...`
// branch when:"ctx...." raw string           -> eq/gt/lt(handle.field, value)
// $.error.* / ctx.error.*                   -> catch (err) => ... err.*
```

TAKEAWAYS

- The old surface exposed the runtime's plumbing: ctx.state and ctx.vars are the same object, ctx.prev and ctx.response are the same object, and the \$ proxy is just typed sugar that deep-walks to js/ctx.* strings at definition time (unwrapProxies in core/workflow-helper/src/proxy/\$.ts) — an author had to learn three names for one thing and two ways to write each. The proxy also emits what you type verbatim (\$.req -> js/ctx.req), so the aliases leak into the wire strings too.
- ctx.prev only holds the IMMEDIATELY previous step and is overwritten every step, so any non-adjacent read forced authors into ctx.state.<id> with a step-id string they had to keep in sync by hand — and to remember each node's output sits under a .data envelope. A handle is shaped like the node's Zod output (no .data) and makes far-upstream reads identical to adjacent ones: you just reference the const.

- The worst footgun is branch when: it is a RAW ctx string evaluated directly against ctx, never mapper-resolved and never typechecked (the shipped workflows/json/countries-vs-facts.json uses when: 'ctx.request.query.countries === "true"'). Write \$.state.x there and it compiles to a js/ctx string the condition evaluator ignores, silently evaluating to undefined. eq/gt/lt (already in @blokjs/helper) replace it with a compiler-checked typed op.
- Nothing changes underneath: every step's output is still auto-persisted to state[id] on success, and the mapper still resolves a structural reference against state[step] before the consuming step runs — same engine. The handle is just the typed, named surface over that existing auto-persistence: no manual save step, and ephemeral:true still opts a side-effect-only step out.
- Net author-visible surface drops from {ctx.request, ctx.req, ctx.prev, ctx.response, ctx.state, ctx.vars, ctx.error, \$, js/ strings, .data envelopes, raw when} to exactly {step handles, req, tpl, eq/gt/lt, forEach}. No \$, no js/, no ctx anywhere in authoring.

05 Persistence: save every response, use it anywhere

The founder question is "can I save any node's response as a variable and use it at any point later?" Blok already answers yes at the engine level: every step's output is auto-persisted to `state[id]` on success (`PersistenceHelper.applyStepOutput`, Rule 3), and that slot is referenceable anywhere downstream — not just the adjacent step. The redesign keeps that exact default and exact mechanic: a `{ $ref }` resolved against `state[step]` by the same Mapper before the consuming step runs. What changes is only the surface — instead of the stringly-typed `$.state.<id>` you hold a typed handle returned by `step()`, shaped like the node's Zod output. Same persistence, same resolution engine, a surface the compiler can check.

- Save a node response once, read it 4 steps later (the founder question, answered)

● BEFORE — today

Auto-persist already lets any step read any earlier step — but every reference is an unchecked string

```
import { workflow, $ } from "@blokjs/helper";

// OLD: every step auto-persists to ctx.state[id] on success
// (PersistenceHelper Rule 3). You reference an earlier output
// anywhere downstream as the stringly-typed "$.state.<id>" — the
// loader rewrites "$." to "js/ctx." and the Mapper resolves it before
// the consuming step runs. Two reference forms coexist:
//   - a bare ref:      "$.state.validate.qty"
//   - mid-string interp: must drop to a js/ template, because a $-ref
//     can't be embedded inside a URL string in JSON/TS.
// Nothing is typed: a typo in "$.state.valdate" or a wrong field path
// passes lint + typecheck and only fails at runtime (with
// BLOK_MAPPER_MODE=strict, the default, it's a fail-fast step error).
export default workflow({
  name: "Order Intake",
  version: "1.0.0",
  trigger: { http: { method: "POST", path: "/orders" } },
  steps: [
    {
      id: "validate",
      use: "validate-order",
      inputs: { body: $.req.body },
```

```

    },
    {
      id: "customer",
      use: "@blokjs/api-call",
      // mid-string interp -> js/ template (no $-ref form available here)
      inputs: { url: "js/`https://crm.api/customers/${ctx.state.validate.customerId}`" },
    },
    {
      id: "checkStock",
      use: "@blokjs/api-call",
      inputs: { url: "js/`https://inv.api/stock/${ctx.state.validate.productId}`" },
    },
    {
      id: "price",
      use: "compute-price",
      inputs: { qty: "$.state.validate.qty", tier: "$.state.customer.tier" },
    },
    {
      id: "createOrder",
      use: "create-order",
      // Reaches back to `validate` (3 steps up) AND `customer` AND `price`.
      // Auto-persistence means none of these needed a manual "save" step.
      inputs: {
        productId: "$.state.validate.productId",
        qty: "$.state.validate.qty",
        customerId: "$.state.customer.id",
        amount: "$.state.price.total",
      },
    },
    {
      id: "ok",
      use: "@blokjs/respond",
      inputs: { status: 201, body: "$.state.createOrder" },
      ephemeral: true,
    },
  ],
});

```

● AFTER — proposed

Same auto-persist, same `{ $ref } → state[id]` resolution — now every cross-step reference is a typed handle the compiler checks

```
import { workflow, step, http, tpl, respond } from "@blokjs/core";
import { validateOrder, computePrice, createOrder } from "../nodes/order";
import { apiCall } from "@blokjs/nodes-http";

// NEW: SAME engine. Every step still auto-persists to state[id] on
// success; the result is still referenceable anywhere downstream.
// step("id", node, inputs) returns a TYPED HANDLE shaped like the
// node's Zod OUTPUT. Reading `validate.productId` records a structural
// ref { $ref: { step: "validate", path: ["productId"] } } that the
// Mapper resolves against state["validate"] before `createOrder` runs.
// `tpl` is a tagged template so refs stay structural inside a string –
// it replaces the mid-string js/ template, no js/ or ctx anywhere.
export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  const validate = step("validate", validateOrder, { body: req.body });

  const customer = step("customer", apiCall, {
    url: tpl`https://crm.api/customers/${validate.customerId}`,
  });

  const stock = step("checkStock", apiCall, {
    url: tpl`https://inv.api/stock/${validate.productId}`,
  });

  const price = step("price", computePrice, {
    qty: validate.qty,
    tier: customer.tier,
  });

  // Reaches back to `validate` (the handle from 3 steps up) AND `customer`
  // AND `price`. No save step, no "$.state.*" strings – every reference is
  // the const you already hold, and `amount` is type-checked against
  // computePrice's output shape. A typo'd field is a compile error.
  const order = step("createOrder", createOrder, {
    productId: validate.productId,
```

```

    qty: validate.qty,
    customerId: customer.id,
    amount: price.total,
  });

  return step("ok", respond, { status: 201, body: order }, { ephemeral: true });
});

```

● ephemeral opt-out + the persistence knobs (as / spread) carry over unchanged

● BEFORE — today

ephemeral / spread / as are per-step string flags on an untyped step object

```

import { workflow, $ } from "@blokjs/helper";

// OLD persistence knobs (PersistenceHelper.applyStepOutput, in order):
// Rule 0 errored result -> NO write (truthful "did it succeed?")
// Rule 1 ephemeral:true -> NO write (only ctx.prev carries it)
// Rule 2 spread:true     -> shallow-merge result.data keys into state
//                        (NO state.load slot - keys land at top level)
// Rule 3 default        -> state[as ?? id] = result.data
export default workflow({
  name: "Sync Inventory",
  version: "1.0.0",
  trigger: { http: { method: "POST", path: "/sync" } },
  steps: [
    {
      id: "audit",
      use: "audit-log",
      inputs: { event: "sync.started", at: "$.req.body.ts" },
      ephemeral: true, // side-effect only: never lands in state (Rule 1)
    },
    {
      id: "load",
      use: "fetch-user-and-profile",
      spread: true, // result.data = { user, profile } -> state.user + state.profile
    },
  ],
});

```



```

{
  id: "rawReport",
  use: "build-report",
  // reads the SPREAD keys directly off state – not state.load.user
  inputs: { user: "$.state.user", profile: "$.state.profile" },
  as: "report", // store at state.report instead of state.rawReport (Rule 3)
},
{
  id: "respond",
  use: "@blokjs/respond",
  // `audit` is NOT in state (ephemeral); `report` is the renamed slot;
  // `user` came from spread. All resolved from state by the Mapper.
  inputs: { body: { report: "$.state.report", user: "$.state.user" } },
  ephemeral: true,
},
],
});

```

● AFTER — proposed

Same Rule 0–3 engine; ephemeral / spread / as move to a typed options bag, no behavior change

```

import { workflow, step, http, respond } from "@blokjs/core";
import { auditLog, fetchUserAndProfile, buildReport } from "../nodes/sync";

// NEW: IDENTICAL persistence rules – the knobs are the SAME flags, now
// in an options bag. Default is still auto-persist to state[id]; the
// handle is the typed view of that slot. `ephemeral` opts a side-effect
// step out (no slot, no handle to read). `as` renames the slot;
// `spread` flattens a multi-output node – so its handle exposes the
// spread keys directly (load.user, load.profile), mirroring how spread
// lands at the top level of state. as/spread stay mutually exclusive,
// enforced at load time exactly as before.
export default workflow("sync-inventory", { trigger: http.post("/sync") }, (req) => {
  // side-effect only: ephemeral -> never written to state, returns no handle
  step("audit", auditLog, { event: "sync.started", at: req.body.ts }, { ephemeral: true });

  // multi-output node: spread flattens result.data -> state.user + state.profile.

```

```

// the handle exposes those same keys: load.user / load.profile.
const load = step("load", fetchUserAndProfile, {}, { spread: true });

// rename the persisted slot: stored at state.report; handle still typed.
const report = step("report", buildReport, {
  user: load.user,
  profile: load.profile,
});

// `audit` produced no handle (ephemeral); `report` and the spread
// keys are read straight off their typed handles.
return step("respond", respond, {
  body: { report, user: load.user },
}, { ephemeral: true });
});

```

TAKEAWAYS

- The founder ask — 'save any node response as a variable, usable at any point' — is already how Blok works: applyStepOutput auto-persists every step's output to state[id] on success (Rule 3). Errored steps write nothing (Rule 0), so state[id] === undefined is a truthful 'did it succeed?' check. The redesign keeps this default verbatim; there is no manual save step in either model.
- 'Use it anywhere' is literal, not adjacent-only. ctx.prev is just the previous step; cross-step access goes through state[id] — which the new createOrder step demonstrates by reaching a handle 3 steps back, alongside two intermediate ones.
- The mechanic is unchanged: reading a handle (validate.productId) records { \$ref: { step, path } }; the same Mapper that resolves '\$.state.*' today resolves that ref against state[step] before the consuming step runs. New surface, identical resolution engine.
- Only the surface changes — stringly-typed '\$.state.<id>' (and mid-string 'js/ ...' templates, now tpl) become typed handles shaped like the node's Zod output. A wrong step id or field path is a compile error instead of a runtime MapperResolutionError.
- The persistence knobs carry over one-for-one: ephemeral opts a side-effect step out of state, spread flattens a multi-output node's keys to the top level (so its handle exposes load.user / load.profile, not load), as renames the slot, as/spread stay mutually exclusive. They move from per-step string flags to a typed options bag with no behavior change.
- Why keep auto-persist-all rather than opt-in saves: opt-in would reintroduce the manual 'save this' step the founder is trying to escape, and would make the truthful-state contract (Rule 0) unreliable. Auto-persist-all + typed handles gives free cross-step access AND compile-time safety — the alternatives trade one away.

06 Cross-runtime nodes: from stringly-typed runtime refs to generated typed stubs

Blok's superpower is that a node can run in any of 7 runtimes (Go, Rust, Java, C#, PHP, Ruby, Python) over one gRPC contract (`blok.runtime.v1`). But authoring a workflow against a Python node today means hand-writing a magic pair — `use: "risk-score" + `type: "runtime.python3"` — registering the local TS nodes through a `Nodes.ts` map, and trusting that the field names you pass match a schema written in another language. There is zero type-safety across the boundary. The new model generates a typed stub from each node's JSON Schema — the very `input_schema_json` / `output_schema_json` the runtime already returns from its `ListNodes` RPC (`NodeDescriptor` in runtime.proto) — so a Python node is imported and called exactly like a local TS node: same handle, same autocomplete, same IR. `defineNode()` in each SDK and the gRPC wire contract are untouched — only the authoring surface changes.

● Calling a Python node from a TS workflow — stringly-typed runtime ref + Nodes.ts map

● BEFORE — today

`use` + `type: "runtime.python3"` are opaque strings; Nodes.ts is a hand-kept map; nothing type-checks across the language boundary

```
// triggers/http/src/Nodes.ts
// Hand-maintained map of every node the trigger can resolve in-process.
// Cross-runtime nodes are NOT here — they're reached only by the magic
// `type: "runtime.<kind>"` string in the workflow step (see below).
import type { BlokService } from "@blokjs/runner";
import validateOrder from "../nodes/validate-order/index"; // local TS node
import createOrder from "../nodes/create-order/index";      // local TS node

const nodes: Record<string, BlokService<unknown>> = {
  "validate-order": validateOrder,
  "create-order": createOrder,
};

export default nodes;

// triggers/http/src/workflows/orders/intake.workflow.ts
```

```

import { workflow, $ } from "@blokjs/helper";

export default workflow({
  name: "Order Intake",
  version: "1.0.0",
  trigger: { http: { method: "POST", path: "/orders" } },
  steps: [
    {
      id: "validate",
      use: "validate-order",          // resolved via Nodes.ts -> in-process
      inputs: { body: "$.req.body" },
    },
    {
      // The Python node. No type-checking of inputs or outputs – these are
      // opaque strings. A typo in `use`, a wrong field name, or a schema drift
      // in the Python node surfaces only at run time as a gRPC error.
      id: "score",
      use: "risk-score",              // must match a name in the python3 registry
      type: "runtime.python3",        // magic string -> GrpcRuntimeAdapter(python3)
      inputs: {
        productId: "$.state.validate.productId",
        qty: "$.state.validate.qty",
        customerTier: "$.req.body.tier", // is `tier` even a field? nobody checks
      },
    },
    {
      id: "respond",
      use: "@blokjs/respond",
      inputs: {
        status: 201,
        // `$.state.score.risk` – `risk`? `riskScore`? `score`? Whatever the
        // Python node happens to return. No editor help, no compile error.
        body: { orderId: "$.state.validate.productId", risk: "$.state.score.risk" },
      },
    },
  ],
});

```

● AFTER — proposed

the Python node is a generated typed stub, imported and called exactly like a local node — full autocomplete and compile-time checks across the language boundary

```
// nodes/risk-score.python.ts — GENERATED from the node's JSON Schema manifest
// (the same input_schema_json / output_schema_json the python3 runtime returns
// from its ListNodes RPC — see runtime.proto NodeDescriptor).
// `blokctl nodes sync` regenerates this whenever the Python node's schema
// changes. Do not edit by hand.
import { runtimeNode } from "@blokjs/core";

export const riskScore = runtimeNode<
  { productId: string; qty: number; customerTier: string },
  { risk: number; band: string; reasons: string[] }
>(
  "risk-score",          // node name, as registered in the python3 runtime
  "runtime.python3",     // the runtime kind -> GrpcRuntimeAdapter(python3) at run time
);

// triggers/http/src/workflows/orders/intake.workflow.ts
import { workflow, step, http, respond } from "@blokjs/core";
import { validateOrder } from "../../nodes/validate-order"; // local TS node, imported directly
import { riskScore } from "../../nodes/risk-score.python"; // Python node, imported like any node

// No Nodes.ts. Local nodes are plain imports; the Python node is its typed stub.
export default workflow("order-intake", { trigger: http.post("/orders") }, (req) => {
  const validate = step("validate", validateOrder, { body: req.body });

  // Calling the Python node is indistinguishable from calling a local node.
  // validate.productId / validate.qty are typed handles; req.body.tier is typed
  // from the trigger. A wrong field name is a COMPILE error, not a 3am gRPC error.
  const score = step("score", riskScore, {
    productId: validate.productId,
    qty: validate.qty,
    customerTier: req.body.tier,
  });

  // score.risk / score.band are typed from the Python node's output schema.
```

```

return step("ok", respond, {
  status: 201,
  body: { orderId: validate.productId, risk: score.risk, band: score.band },
});
});

```

- **Per-runtime registration — codegen shim (compiled) vs fs-scan (dynamic), now uniform**

- **BEFORE — today**

two divergent registration paths — codegen shim that recompiles every dev-spawn (Go/Rust/Java/C#) vs fs-scan with no type surface (Python/Ruby/PHP) — and no node schema ever reaches the TS author

```

// packages/cli/src/services/runtime-setup.ts (shape of today's setup)
//
// Registration is DIFFERENT per runtime family. The CLI has to know which
// mechanism each of the 7 runtimes uses.
//
// COMPILED (Go, Rust, Java, C#): a registration shim is CODEGEN'd into the SDK
// build and recompiled on every `blokctl dev`. For Go, generateGoNodeRegistry()
// scans runtimes/go/nodes, copies each into the SDK module under usernodes/,
// and emits cmd/server/register_user_nodes.go:

```

```

function generateGoNodeRegistry(nodeNames: string[]): string {
  const importLines = [
    '\tblok "github.com/nickincloud/blok-go"',
    ...nodeNames.map(
      (n, i) => `\tusernode${i} "github.com/nickincloud/blok-go/usernodes/${n}"`,
    ),
  ];
  const callLines = nodeNames.map((_, i) => `\tusernode${i}.Register(registry)`);
  return [
    '// Code generated by blokctl. DO NOT EDIT.',
    'package main',
    '',
    'import (',
    importLines.join("\n"),
    ')',
  ].join("\n");
}

```

```

    "",
    "// AUTO-GENERATED on every dev spawn. A syntax error in any user node",
    "// fails the Go BUILD silently – the supervisor sees a crash, not the CLI.",
    "func registerUserNodes(registry *blok.NodeRegistry) {",
    callLines.join("\n"),
    "}",
  ].join("\n");
}

// DYNAMIC (Python, Ruby, PHP): no codegen, no build. The SDK fs-scans a
// directory at boot. Type errors are caught only at run time.
function wireDynamicRuntime(kind: "python3" | "ruby" | "php", env: EnvFile): void {
  // -> .env.local
  env.set(`RUNTIME_${kind.toUpperCase()}_HOST`, "127.0.0.1");
  env.set(`RUNTIME_${kind.toUpperCase()}_GRPC_PORT`, defaultGrpcPort(kind));
  env.set("BLOK_NODES_DIR", `runtimes/${kind}/nodes`); // SDK fs-scans this at boot
  // No schema is surfaced to the TS side. The workflow author learns the node's
  // shape by reading the Python source, or by trial and error.
}

// And the runner, at boot, registers one gRPC adapter per kind (Configuration.ts):
for (const kind of ["go", "rust", "java", "csharp", "php", "ruby", "python3"]) {
  registry.register(buildGrpcAdapter(kind, hostFor(kind), grpcPortEnvFor(kind)));
}

```

● AFTER — proposed

one `blokctl nodes sync` reads each runtime's existing ListNodes manifest and emits a typed stub — identical for all 7 runtimes; the IR and gRPC dispatch are unchanged

```

// One uniform pipeline for ALL 7 runtimes. The runtime already knows its nodes
// and their schemas – it returns them from the ListNodes RPC (runtime.proto:
// NodeDescriptor { name, input_schema_json, output_schema_json }). We read that
// manifest and generate a typed stub. No per-family special-casing.
//
// $ blokctl nodes sync
// ✓ go      4 nodes -> nodes/*.go.ts
// ✓ rust    2 nodes -> nodes/*.rust.ts

```

```

// ✓ java      1 node  -> nodes/*.java.ts
// ✓ csharp    1 node  -> nodes/*.csharp.ts
// ✓ php       3 nodes -> nodes/*.php.ts
// ✓ ruby      2 nodes -> nodes/*.ruby.ts
// ✓ python3   5 nodes -> nodes/*.python.ts
//
// packages/cli/src/services/nodes-sync.ts (one path for every kind)
import { runtimeRegistry, type NodeDescriptor } from "@blokjs/core";
import { jsonSchemaToTs } from "../schema-codegen";

export async function syncNodes(): Promise<void> {
  for (const kind of runtimeRegistry.kinds()) {
    const adapter = runtimeRegistry.adapter(kind);
    const { nodes } = await adapter.listNodes();
    for (const node of nodes) {
      await writeStub(kind, node);
    }
  }
}

async function writeStub(kind: string, node: NodeDescriptor): Promise<void> {
  const In = jsonSchemaToTs(node.inputSchemaJson);
  const Out = jsonSchemaToTs(node.outputSchemaJson);
  const ident = camelCase(node.name);
  const file = `nodes/${node.name}.${stubExt(kind)}.ts`;
  await write(
    file,
    `// GENERATED from ${kind} ListNodes manifest. Do not edit. Run `blokctl nodes sync`.
    `import { runtimeNode } from "@blokjs/core";
    `export const ${ident} = runtimeNode<${In}, ${Out}>
    `  ${JSON.stringify(node.name)},
    `  ${JSON.stringify("runtime." + kind)},
    `);
  );
}

// At run time NOTHING changed: the typed stub compiles to the SAME language-

```



```

// neutral IR the runner already executes, and the SAME GrpcRuntimeAdapter
// dispatches it over gRPC. The stub is a compile-time convenience; the wire and
// the { use, type } the runner resolves are untouched.
//
// step("score", riskScore, { productId: validate.productId, ... })
//   --compiles to-->
//   {
//     "id": "score", "use": "risk-score", "type": "runtime.python3",
//     "inputs": {
//       "productId": { "$ref": { "step": "validate", "path": ["productId"] } }
//     }
//   }

```

TAKEAWAYS

- The runtime ref is no longer a string. `use: "risk-score" + `type: "runtime.python3"` becomes a generated, typed `runtimeNode<In, Out>` stub, imported and called like any local TS node — a wrong field name or stale schema is a compile error, not a 3am gRPC failure.
- Nodes.ts is deleted. Today's `triggers/http/src/Nodes.ts` (`Record<string, BlokService<unknown>>`) is replaced by plain imports for local TS nodes and generated stubs for cross-runtime nodes. There is no hand-maintained node map to drift out of sync.
- Registration is uniform across all 7 runtimes. The old split — codegen shim recompiled every dev-spawn (Go/Rust/Java/C#, e.g. `register\_user\_nodes.go`) vs fs-scan with no type surface (Python/Ruby/PHP via `BLOK\_NODES\_DIR`) — collapses into one `blokctl nodes sync` that reads each runtime's existing `ListNodes` manifest.
- Nothing on the wire changes. `defineNode()` in each SDK and the `blok.runtime.v1` gRPC contract (Execute / ListNodes / NodeDescriptor with `input\_schema\_json` / `output\_schema\_json`) are untouched. The stub is pure compile-time ergonomics over the schema the runtime already publishes.
- Same IR, same engine. `step(...)` against a Python node compiles to the identical language-neutral `{ use, type: "runtime.python3", inputs: { \$ref } }` IR, resolved by the same mapper and dispatched by the same `GrpcRuntimeAdapter` as today.

Migration & compatibility

How existing projects move — hybrid, backward-compatible, with a codemod.

- Hybrid by construction: the new DSL and the old v1/v2 array-of-steps shape both compile to the same JSON IR, so a project can hold handle-style and string-style workflows side by side and run them on one runner — no big-bang rewrite.
- Back-compat shims: old `@blokjs/runner` / `@blokjs/shared` / `@blokjs/helper` names stay as deprecated re-exports from `@blokjs/core`, so existing imports keep resolving; the only forced edit in a node file is the import source.
- Nodes.ts removal is mechanical: a codemod walks each workflow's **use:** strings, resolves them against the old registry map, rewrites local refs to direct imports + `step(node,...)` calls, and replaces runtime refs (use+type pair) with generated runtimeNode stubs from **blokctl nodes sync** (reads each runtime's existing ListNodes manifest).
- Ref codemod: **`$.state.` / `js\...\`` / `raw-ctx` when **strings rewrite to handle reads / tpl / eq-gt-lt ops; unresolvable expressions (dynamic paths) are left as-is with a `// ponytail: hand-migrate`` marker rather than guessed.****
- What breaks (intentionally, and only at the typed surface): a wrong step id, deleted node, or renamed output field that used to fail at runtime now fails at compile time — louder, earlier, before save. Runtime behavior of correct workflows is unchanged.
- Lockstep release ends post-migration: framework ships as one `@blokjs/core` version and each node moves on independent semver, so a runner patch no longer forces a coordinated re-release of every node package.

Design notes: eq/gt/lt are described in two roles across sections (today's "escape hatch" vs the "new typed ops"). Source confirms today's versions return plain ``string`` over the `$` proxy — so they genuinely change, not just get reused. Resolution: the framing states the new ops read real handle fields (compile-checked) where today's return an untyped string builder; both claims hold and are not contradictory. · The persistence and context sections both assert "nothing changes underneath." Verified true against `applyStepOutput` and the Mapper — the only divergence is surface, so I kept a single consistent statement ("identical runtime, checkable surface") rather than letting each section imply a different degree of change. · The runtimeNode stub is sourced from the node manifest. Verified the manifest is the ListNodes RPC's `input_schema_json/output_schema_json` in `runtime.proto` — consistent across the nodes and cross-runtime sections; no drift between "manifest" and "JSON Schema" wording (they are the same bytes). · `ephemeral/as/spread` are described as carried-over knobs in both persistence and context sections; `applyStepOutput` confirms all three still branch there. Consistent — they move from string flags to a typed options bag with no behavior change.